

MANUAL TÉCNICO · SITICKETS

Arquitectura y Operación de SITickets

Referencia para quien instale, mantenga o extienda el sistema: arquitectura, código, API, modelo de datos y despliegue.

Frontend Angular 22 Backend NestJS 11 Base de datos MySQL 8 Conexiones 4

Congreso del Estado de México — Secretaría de Administración y Finanzas

Dirección de Informática · Departamento de Soporte Técnico y Telecomunicaciones

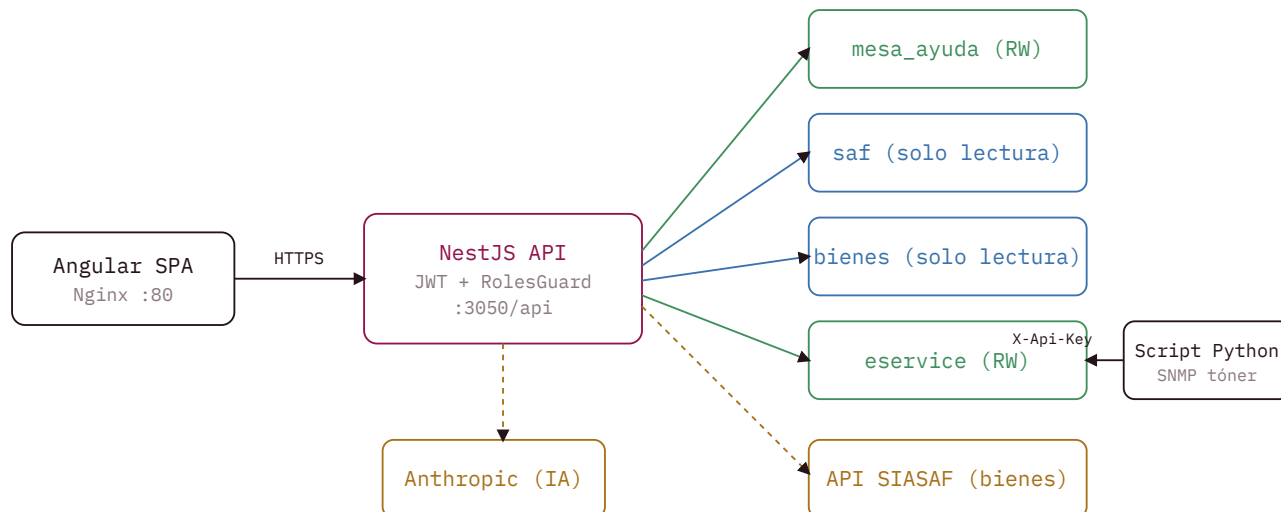
“2026. Año del Humanismo Mexicano en el Estado de México”.

Contenido

- | | |
|-------------------------|---------------------------|
| 1 Arquitectura general | 7 Modelo de datos |
| 2 Stack tecnológico | 8 Motor de reglas |
| 3 Estructura del código | 9 Variables de entorno |
| 4 Rutas del frontend | 10 Despliegue |
| 5 Autenticación y roles | 11 Seguridad y pendientes |
| 6 API REST | |

1 Arquitectura general

SPA de Angular servida por Nginx, que consume una API REST de NestJS. El backend mantiene **cuatro conexiones simultáneas a MySQL 8**: una propia de escritura y tres bases externas que ya operaban en la institución, a las que el sistema se conecta sin modificarlas (salvo la de impresoras).



La API es el único punto que toca las cuatro bases de datos; el frontend solo habla con la API.

2 Stack tecnológico

Frontend

Angular 22 (standalone components, signals, control flow @if/@for), TypeScript 6, Bootstrap 5.3 + Bootstrap Icons, RxJS 7.8, Vitest.

Backend

NestJS 11, Sequelize + sequelize-typescript, mysql2, Passport-JWT, bcrypt, Puppeteer (PDF), class-validator.

Datos

MySQL 8, cuatro conexiones: mesa_ayuda, saf, bienes, eservice.

Despliegue

Docker multi-stage, Nginx como servidor estático del SPA, Chromium del sistema para Puppeteer.

3 Estructura del código

Frontend — src/app

- core/ — capa transversal: api.ts (URL base por entorno), auth.service.ts, auth.interceptor.ts, guards.ts (guardSesion, guardAnonimo, guardRol), modelos.ts, tickets.service.ts, usuarios.service.ts, formato.ts.
- comp/ — componentes compartidos: tabla-tickets.ts, detalle-ticket.ts (el más grande, concentra todo el ciclo de vida).

- paginas/ — una carpeta por pantalla: login, shell, tickets, bandeja, monitor, tablero, disponibilidad, internos, catalogo-servicios, catalogo-problemas, prioridades, usuarios, y solicitante/ (nuevo, mis-tickets).

Es una app standalone (sin NgModules); cada ruta carga su componente con `loadComponent`. No existe una librería de componentes tipo Angular Material: la UI está construida sobre Bootstrap.

Backend — backend/src

- auth/ — login, perfil, cambio de contraseña, JwtStrategy, JwtAuthGuard, RolesGuard.
- tickets/ — el núcleo: tickets.service.ts (ciclo de vida completo), reglas.service.ts (motor de folio / asignación / escalamiento), reloj.service.ts, traza.service.ts, dictamen.service.ts (PDF con Puppeteer), cedula-custodia.service.ts.
- catalogos/, usuarios/, bienes/, impresoras/, ia/, operacion/ (monitor, tablero, calendario, mantenimiento preventivo).
- database/ — modelos Sequelize y migraciones.service.ts.seed/ — siembra inicial de catálogos y padrón. common/ — decoradores @Roles, @Publico, @UsuarioActual, ApiKeyGuard.

4 Rutas del frontend

RUTA	PANTALLA	ROLES CON ACCESO
/entrar	Login	anónimo
/nuevo	Registrar ticket	todos los roles autenticados
/mis-tickets	Mis tickets	solicitante, admin, gestor, tecnico, jefe, proveedor
/tickets	Todos los tickets	admin, operador
/bandeja	Bandeja	tecnico, jefe, proveedor
/monitor	Monitor	admin, jefe, tecnico, proveedor, operador
/tablero	Tablero	admin, jefe, operador
/disponibilidad	Disponibilidad	admin
/internos	Tickets internos	admin, jefe
/catalogo-servicios	Catálogo de servicios	admin
/catalogo-problemas	Catálogo de problemas	admin
/prioridades	Prioridades	admin
/usuarios	Usuarios	admin

El guard de rol en el frontend (`guardRol`, en `core/guards.ts`) es una comodidad de navegación; la seguridad real se revalida en cada petición contra el `RolesGuard` del backend.

5 Autenticación y roles

1. El usuario envía RFC y contraseña a `POST /auth/login` (limitado a 5 intentos por minuto).
2. El backend compara contra `Usuario.password_hash` (personal interno) o contra la contraseña real del padrón saf (solicitantes externos), siempre con `bcrypt` costo 12 — incluso cuando el RFC no existe, para no filtrar por tiempo de respuesta qué cuentas son válidas.
3. Se emite un JWT (vigencia configurable, 8 h por omisión) que el frontend guarda y envía como `Authorization: Bearer`.
4. `JwtStrategy` revalida al usuario en **cada petición**, no solo al iniciar sesión: si es personal interno, contra `Usuario.activo`; si es externo, contra el estado en el padrón saf.
5. Guards globales, en este orden: `JwtAuthGuard` (exige token salvo en rutas `@Publico()`) → `RolesGuard` (exige que el rol esté en `@Roles(...)` si el endpoint lo declara) → `ThrottlerGuard`.

Un caso especial: `ApiKeyGuard` autentica por cabecera `X-API-Key` en lugar de JWT, para las dos rutas que llama el script Python de lectura de tóner.

6 API REST

Todas las rutas cuelgan del prefijo global `/api`.

Autenticación

POST <code>/auth/login</code>	Público, 5/min	Iniciar sesión
GET <code>/auth/yo</code>	Sesión	Perfil del usuario autenticado
POST <code>/auth/password</code>	Sesión (no externos)	Cambiar contraseña

Tickets

GET /tickets	Listado acotado por rol	
GET /tickets/solicitantes	admin, operador, gestor	Buscar solicitante en padrón SAF
GET /tickets/:id	Detalle de un ticket	
GET /tickets/:id/bien	Bien mueble asociado	
GET /tickets/:id/nivel-toner	Nivel de tóner de la impresora asociada	
POST /tickets	Crear ticket	
POST /tickets/:id/datos	Corregir datos generales	
POST /tickets/internos	admin	Alta de ticket interno
POST /tickets/:id/iniciar espera reanudar	técnico asignado	Transiciones de atención
POST /tickets/:id/resolver	técnico asignado	Resolver con diagnóstico y solución
POST /tickets/:id/atender-cmp	técnico de cómputo	Multipart, hasta 12 fotos (JPG/PNG, 20 MB)
POST /tickets/:id/validar rechazar reabrir cancelar	solicitante	Cierre del ciclo
POST /tickets/:id/reasignar	admin, operador	Cambiar técnico (motivo obligatorio)
POST /tickets/:id/reclasificar	admin	Cambiar servicio/problema
POST /tickets/:id/prioridad	admin	Cambiar prioridad
GET /tickets/:id/dictamen cedula-salida cedula-entrada	Descarga de PDF	
POST /tickets/:id/reloj/inicio fin	técnico	Reloj checador

Catálogos, usuarios y otros módulos

GET /catalogos	Catálogos combinados para formularios	
GET /catalogos/organizacion	Público	Dependencias / áreas / sedes
POST / PATCH /catalogos/problemas...	admin	CRUD del catálogo de 66 problemas
POST / PATCH /catalogos/servicios...	admin	CRUD de servicios y usuarios permitidos
PATCH /catalogos/prioridades/:clave	admin	Tiempos objetivo por prioridad
GET /usuarios/saf · GET / POST /usuarios · PATCH /usuarios/:id	admin	Padrón, alta y edición de personal interno
GET /bienes/mios · /bienes/de/:idUsuarioSaf	propio / admin, operador, gestor	Bienes bajo resguardo
GET /impresoras/modelos	Catálogo de modelos de impresora	
POST /pythonCompara · /pythonNotificaError	X-Api-Key	Ingesta de lecturas de tóner (script Python/SNMP)
POST /ia/mejorar-observaciones	roles CMP	Redacción asistida del dictamen
GET /monitor · /tablero · /calendario · /traza	Pantallas de operación · POST /calendario para bloquear disponibilidad	

7 Modelo de datos

17 modelos Sequelize, repartidos en las cuatro conexiones. El sistema no crea bases de datos: se conecta a las que ya operan en la institución (salvo eservice, donde también escribe).

mesa_ayuda — lectura/escritura, propia del sistema

- **Organización:** Dependencia, Area, Sede (con coordenadas, usadas por el reloj checador).

- **Cuentas:** Usuario (rol ENUM de 7 valores, password_hash, activo).
- **Catálogos:** Servicio, ServicioUsuarioPermitido, TecnicoServicio, Prioridad, Estatus, CatalogoProblema, MotivoReasignacion, FolioSerie.
- **Núcleo:** Ticket (folio vigente vs. folio_general inmutable, fechas de todo el ciclo de vida, contadores de reasignaciones/reaperturas/rechazos, campos de dictamen y cédulas), TicketTecnico (equipo en internos), TicketSesion (reloj checador, con lat/lng/exactitud/distancia), TicketBitacora (auditoría), CalendarioTecnico, ProgramaPreventivo.

Bases externas

- saf (solo lectura) — UserSaf (RFC/contraseña reales), SUsuario, SDependencia, SDireccion, SDepartamento, SUBicacion (Departamento): padrón institucional completo.
- bienes (solo lectura) — BienMueble, EstatusBien, ServidorBien: inventario y resguardos, usados para el servicio de equipo de cómputo.
- eservice (lectura y escritura) — Impresora, ComparacionImpre: historial de tóner alimentado por el script Python externo.

Relaciones clave: Ticket referencia a Servicio (vigente y original), CatalogoProblema, Usuario (solicitante / técnico / quien registró) y Dependencia/Area/Sede; TicketBitacora, TicketSesion y TicketTecnico cuelgan siempre de un Ticket.

8 Motor de reglas

Concentrado en tickets/reglas.service.ts:

- **Folio:** serie por prefijo de servicio + año, con bloqueo pesimista transaccional para no duplicar folios bajo concurrencia.
- **Asignación automática:** especialidad → disponibilidad (calendario) → menor carga → rotación justa. Si no hay técnico disponible, el ticket se deja **en cola** en vez de forzar una asignación.
- **Escalamiento automático:** 3 o más tickets del mismo servicio y dependencia en 60 minutos suben todos a prioridad P1.
- **Cierre por omisión:** tarea programada (@Cron) que cierra tickets resueltos sin validar a los 3 días.
- **Trazabilidad:** toda reasignación o reclasificación exige un motivo y queda escrita en TicketBitacora (método anota).
- **Geolocalización:** distancia de Haversine entre la ubicación reportada y la sede esperada, para el reloj checador.

9 Variables de entorno

Definidas en backend/.env.example.

VARIABLE	USO
PORT	Puerto del backend (3050)
DB_HOST / PORT / NAME / USER / PASS	Conexión principal a mesa_ayuda
DB_SYNC	Crea tablas faltantes automáticamente — solo desarrollo
DB_MIGRAR	Aplica los .sql de sql/migraciones, cada uno una sola vez
SEED_ON_BOOT / SEED_PASSWORD	Siembra catálogos y padrón inicial si la base está vacía
CORREO_DOMINIO	Dominio institucional exigido para correos
BIENES_API_URL / _TIMEOUT_MS / _TOKEN / BIENES_API_BASE_URL	API remota SIASAF de inventario (con captura manual como respaldo si no responde)
ESERVICE_DB_HOST / PORT / NAME / USER / PASS	Cuarta conexión: impresoras arrendadas
PYTHON_TONER_API_KEY	Clave fija para el script Python (cabecera X-API-Key)
JWT_SECRET / JWT_EXPIRES	Firma y vigencia del token (8 h por omisión)
CORS_ORIGIN	Orígenes permitidos
ANTHROPIC_API_KEY / ANTHROPIC_MODEL	Botón de IA para mejorar redacción — opcional, el sistema degrada sin romperse si falta

10 Despliegue

Sin Docker

1. Crear la base mesa_ayuda (utf8mb4) y un usuario con permisos (incluye CREATE/ALTER mientras DB_SYNC=true). Si la base ya existía sin autenticación, correr primero 01_preparar_base_existente.sql.

2. Backend: `cd backend && npm install && npm run start:dev` — escucha en :3050/api.

3. Frontend: en la raíz, `npm install && npm start` — escucha en :4200.

Con Docker

- `docker compose up --build` levanta **frontend** (ng serve con recarga en caliente) y **backend** (nest start:dev), sin levantar MySQL: ambos se conectan al servidor MySQL del área.
- `docker-compose.prod.yml` construye ambas imágenes: el **frontend** compila el build de producción de Angular y lo sirve con Nginx en 8087:80 (BASE_HREF=/sitickets/); el **backend** corre en 3032:3050, monta `./backend/storage` y fija el host de la API externa `siasaf.gob.mx`.
- El Dockerfile del backend instala **Chromium** del sistema para que Puppeteer genere los PDF de dictamen sin descargar su propio binario, y copia `sql/` y `assets/` (membrete institucional) fuera de `dist/`, ya que nest build no los incluye.

Cuentas de ejemplo (entorno de pruebas, siembra inicial)

La siembra crea una cuenta por rol con la contraseña definida en SEED_PASSWORD (por omisión Sitickets2026*): administrador, jefe de departamento, técnico de impresoras, técnico de cómputo, proveedor externo y solicitante.

11 Seguridad y pendientes

Respecto al prototipo original (un HTML sin autenticación real), esta versión agrega: autenticación JWT real, validación de entrada con `whitelist`, contraseñas con `bcrypt` costo 12, las antiguas rutinas SQL reescritas como servicios de NestJS con transacciones, cabeceras `helmet`, CORS acotado y limitación de intentos de login.

Antes de pasar a producción:

- Cargar las coordenadas reales de cada sede (para que el reloj checador valide ubicación correctamente).
- Cambiar `JWT_SECRET`, `SEED_PASSWORD` y las credenciales de MySQL de los valores de ejemplo.
- Poner `DB_SYNC=false` una vez estabilizado el esquema.
- Definir política de respaldo para `ticket_bitacora` y `ticket_sesion` (las tablas de mayor crecimiento).
- Si el sistema sale a internet abierto, mover el token de `localStorage` a una cookie `httpOnly`.